

CIFAR-10 Image Classification with Validation-Controlled Model Selection and Batch Normalization Analysis

Dechao Huang 23300660009

June 12, 2026

Abstract

This report studies CIFAR-10 image classification under a validation-controlled experimental protocol, with emphasis on architecture selection, training recipes, final-test discipline, and the optimization effects of Batch Normalization (BN). During development, the official training set is split into 45,000 `train_dev` images and 5,000 `val_dev` images. All model selection, checkpoint selection, and BN analysis are restricted to this development split; the official test set is used only after the replay checkpoints and evaluation policy have been locked. The experiments start from SimpleCNN and VGG-A baselines, then compare CompactResNetV2, PreActResNet-W4/W6, and the W4-derived L3 recipe branch. The validation-selected recipes are retrained on all 50,000 official training images in a full-train replay. Among the locked replay entries, W6 reaches 96.26% official-test accuracy, L3-lowLR reaches 96.10%, and W4 reaches 95.55%. The BN mini-study shows that BN improves validation accuracy and convergence speed in controlled VGG-A experiments, tolerates a higher usable base learning rate under expanded short-run LR stress, and gives more stable behavior in train-mode scale-invariance and local gradient-predictiveness probes. At the same time, the evidence does not support stronger claims that BN narrows the loss envelope for every learning-rate condition or universally reduces gradient Lipschitz estimates from a single VGG-A/CIFAR-10 setting.

Keywords: CIFAR-10, validation protocol, full-train replay, Batch Normalization, loss landscape, gradient predictiveness

1 Introduction and Experimental Setup

This report addresses model selection, final evaluation, and Batch Normalization analysis for CIFAR-10 image classification. CIFAR-10 contains 50,000 official training images and 10,000 official test images across ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Because repeated tuning can easily turn the test set into an implicit validation set, the project uses a strict development protocol: all search and analysis are performed on an internal `train_dev`/`val_dev` split, while the official test set is reserved for a locked held-out evaluation.

The experimental narrative is organized around three questions. First, how do the candidate architectures trade off accuracy, parameter count, memory, and training time? Second, can the final replay checkpoints be traced back to validation-stage decisions rather than test-set feedback? Third, in a controlled VGG-A comparison, how does BN affect training dynamics, learning-rate robustness, scale invariance, and local gradient behavior? The report therefore follows three main lines of evidence:

1. **Classification performance:** SimpleCNN, VGG-A, and CIFAR-PreActResNet candidates are compared by validation accuracy, model size, training cost, and locked full-train replay results.
2. **Model selection:** Architecture screening, recipe ablation, Stage B training, and Stage C-lowLR fine-tuning determine replay stop points before official-test evaluation.
3. **BN mechanism analysis:** Controlled VGG-A experiments examine BN through convergence thresholds, learning-rate stress, local loss geometry, scale invariance, and gradient predictiveness.

1.1 Train/Validation/Test Protocol

All development-stage model selection is based on `train_dev/val_dev`. The official training set is split in a class-balanced way: 4,500 images per class are assigned to `train_dev`, and 500 images per class are assigned to `val_dev`. This gives a 5,000-image validation set while avoiding class-imbalance artifacts in checkpoint ranking.

The `val_dev` split has three roles. It selects checkpoints for each training run, using validation accuracy as the primary criterion and validation loss as a tie-breaker. It supports Pareto comparisons among candidate models such as W4, W6, and CompactResNetV2. It also serves as the evaluation split for the BN mini-study and its probes. The official test set is excluded from all of these development decisions.

In the full-train replay stage, `val_dev` is no longer used for training-time selection. Instead, each validation-selected recipe is retrained from scratch on all 50,000 official training images. The split file remains only as provenance, documenting where the replay stop points came from. The replay training script disables validation and official-test loaders, and the official test is run only by a replay-specific evaluator after the lock file exists and checkpoint hashes match.

The replay protocol is defined by three constraints. First, the W4, W6, and L3 stop points are written into `fulltrain\_replay\_training\_lock.json` before official-test evaluation. Second, L3-fast and L3-lowLR are treated as two pre-specified questions rather than two candidates to choose between after seeing test accuracy. Third, the official-test evaluator accepts only the checkpoints listed in `fulltrain\_replay\_test\_lock.json`; arbitrary checkpoints cannot be passed in at evaluation time.

Table 1 Main experimental settings and replay submission information.

Item	Setting
Dataset	CIFAR-10, 50,000 official train images + 10,000 official test images
Code and weights	https://github.com/adehuang41/deep_learning_pj2 ; https://www.modelscope.cn/models/adehuang41/deep_learning_pj2
Development split	<code>train_dev</code> : 45,000; <code>val_dev</code> : 5,000; split seed = 42
Split file	<code>splits/cifar10_train45000_val5000_seed42.json</code>
Development selection rule	best validation accuracy, validation loss as tie-breaker
Replay training set	all 50,000 official train images, no validation loader during replay
Replay test entries	W4 raw e205; W6 raw e193; L3-fast raw e133; L3-lowLR EMA e230
Replay evaluation	clean/eval transform, no horizontal-flip TTA
Replay lock	<code>results/fulltrain_replay_20260611/protocol/fulltrain_replay_test_lock.json</code>
Replay output	<code>results/fulltrain_replay_20260611/metrics/fulltrain_replay_official_test_results.csv</code>

1.2 Code Structure and Reproducibility

The codebase is organized around models, training, evaluation, and experiment records. Model definitions are under `src/models/`; the standard training loop is in `src/train.py`; data splitting and dataloaders are implemented in `src/data.py`; full-train replay is run by `src/fulltrain_replay.py`; and replay official-test evaluation is handled by `src/fulltrain_replay_official_test.py`. Training scripts live in `scripts/`, configuration files in `configs/`, and outputs are separated into `results/metrics/`, `results/figures/`, `results/tables/`, and `results/protocol/`.

For reproduction, the protocol files are the primary reference. Models can be re-trained, but the official-test numbers reported here should be interpreted as the frozen results associated with the replay lock. If a model is retrained and tested again, that run should be treated as a new experimental version rather than an overwrite of `results/fulltrain_replay_20260611/`. The project does not claim bitwise reproducibility, so

`cuda_benchmark=True` is retained. Fixed seeds and a fixed split are used to make the comparison protocol fair, not to force identical floating-point trajectories.

1.3 Architecture Summary

Table 2 summarizes the architectures used in the report, including convolutional blocks, pooling, classifiers, activations, BN, dropout, and residual connections. SimpleCNN and VGG-A serve as baseline and control models; CompactResNetV2 is an intermediate efficient candidate; and PreActResNet-W4/W6/L3 form the main replay family. L3 has the same structural scale as W4, but differs in the training recipe through weight decay and later checkpoint policy.

Table 2 Architecture summary.

Model	Conv blocks	Pooling	FC / classifier	Activation / BN / Dropout / Residual	Params
SimpleCNN	3 conv layers: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$, all 3×3	MaxPool $\times 3$	Flatten, $2048 \rightarrow 256 \rightarrow 10$	ReLU; no BN; FC dropout 0.3; no residual	0.62M
VGG-A	8 conv layers: 64, 128, 256×2 , 512×4	MaxPool $\times 5$	$512 \rightarrow 512 \rightarrow 512 \rightarrow 10$	ReLU; no BN; FC dropout 0.5; no residual	9.75M
VGG-A BN	Same VGG-A conv layout, conv bias removed	MaxPool $\times 5$	same as VGG-A	ReLU; BN after every conv; FC dropout 0.5; no residual	9.75M
CompactV2	Stem + 4 residual stages, global avg channels 64/128/256/384, blocks 2/2/2/2, ECA-lite		dropout + linear 10-way	SiLU; BN; dropout 0.2 + drop path; residual	7.75M
PreAct W4 / L3	Stem + 3 pre-activation residual groups, depth 28, widths 64/128/256	global avg	linear 10-way	ReLU; pre-activation BN; block dropout 0.3; residual	5.85M
PreAct W6	Same depth as W4, wider residual groups, widths 96/192/384	global avg	linear 10-way	ReLU; pre-activation BN; block dropout 0.3; residual	13.14M

2 Baseline and Controlled VGG-A Models

2.1 SimpleCNN Baseline

SimpleCNN is used as a low-complexity baseline for the fixed CIFAR-10 preprocessing, development split, and validation checkpoint-selection protocol. The model has only about 0.62M parameters, trains the fastest among the compared models, and uses about 0.09 GiB peak memory. Its early locked official-test accuracy is 86.72%.

The baseline learns the main visual patterns in CIFAR-10 but remains limited on fine-grained categories. Its 13.28% error rate is substantially higher than those of the later VGG-A and PreActResNet replay models. This gap shows that, for this task, model capacity, depth, residual connections, and training recipe all matter for final performance.

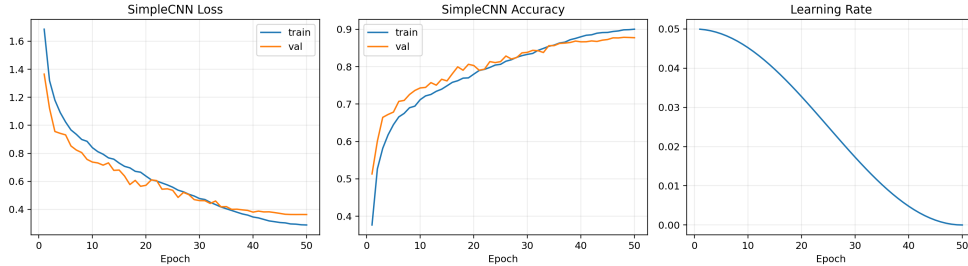


Figure 1 Training and validation curves for the SimpleCNN baseline. The baseline provides a stable low-complexity reference point.

2.2 VGG-A without BatchNorm

VGG-A without BN is the control model for the BatchNorm comparison. It is deeper than SimpleCNN and has about 9.75M parameters, but it contains no BatchNorm layers. Since this VGG-A model has no residual connections, intermediate activations and gradients are more sensitive to scale changes during training; in this setting, the no-BN version converges less effectively than its BN counterpart.

In the early locked official test, VGG-A no BN reaches 87.85% accuracy. This is above SimpleCNN but only by a modest margin, suggesting that simply adding parameters is not enough to guarantee better generalization under a fixed recipe. Its test loss is 0.4783, higher than VGG-A BN’s 0.4291 and higher than every PreActResNet replay model.

2.3 VGG-A with BatchNorm

VGG-A BN is the treatment model in the controlled comparison. It keeps the same backbone layout as VGG-A no BN, with BatchNorm inserted after convolutional layers. This design makes the comparison primarily about normalization rather than architecture. In the early locked official test, VGG-A BN reaches 89.17% accuracy, improving over no-BN by 1.32 percentage points and over SimpleCNN by 2.45 percentage points.

The parameter increase from VGG-A no BN to VGG-A BN is only about 2,752 parameters, negligible relative to a 9.75M-parameter backbone. The performance difference is therefore better explained by the normalization mechanism than by model size. The later BN P0 validation and local-geometry probes are consistent with this interpretation: BN improves VGG-A’s optimization behavior in this project setting.

Table 3 Efficiency and performance of early baseline/control models and the original W4 full-train entry.

Model	Params	Train time	Epoch time	Peak memory	Test acc
SimpleCNN baseline	620,362	530.49 s	10.58 s	0.09 GiB	86.72%
VGG-A no BN	9,750,922	531.92 s	10.39 s	0.27 GiB	87.85%
VGG-A BN	9,753,674	533.16 s	10.41 s	0.33 GiB	89.17%
Original W4 full-train	5,849,050	4950.79 s	23.96 s	3.41 GiB	95.52%

3 Validation-Stage Model Selection and Ablation Path

3.1 Selection Roadmap

Replay stop points are determined through staged validation decisions on `val_dev`, not by comparing official-test results. The selection process has two coordinated tracks. The first is architecture screening, where W4, W6, and CompactResNetV2 are compared to decide which model family should receive longer training. The second is W4 recipe ablation, where the architecture is fixed and only activation, loss, optimizer, or weight decay is changed. Promising short-run branches are then resumed to a 150-epoch budget.

This staged design avoids two sources of confounding. First, a 50-epoch result is not treated as a final conclusion, because early optimizer or regularization advantages can disappear during longer training. Second, architecture changes and recipe changes are not mixed into one leaderboard, which would make it unclear whether a difference comes from model structure or training hyperparameters. All screening uses the same `train_dev/val_dev` split, seed 42, and checkpoint-selection rule, with no official-test access.

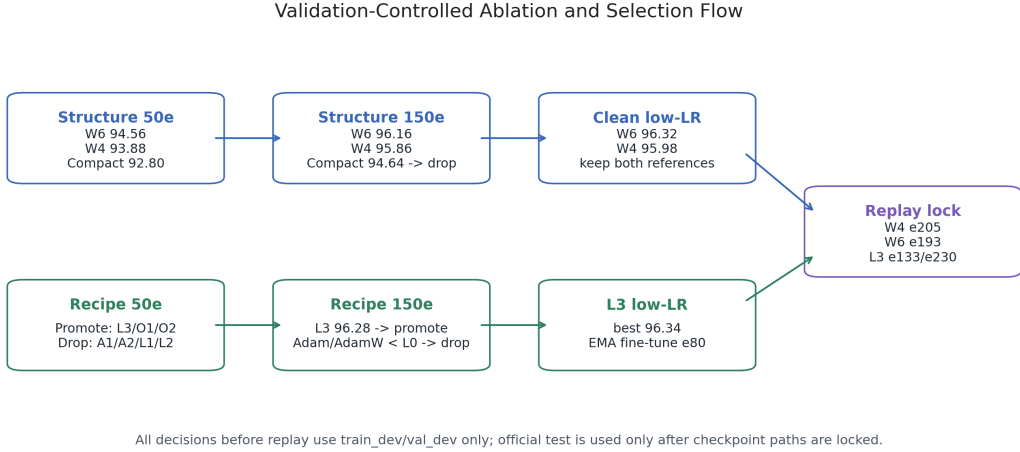
3.2 50-Epoch Recipe Screening

The 50-epoch ablation uses `L0_current_recipe` as the control, reusing the epoch-50 record from W4 Stage B. This reuse is not an arbitrary historical comparison: before using it, I checked that the configuration, split file, seed, scheduler semantics, and dataloader settings matched the new ablation base config. The screen uses `scheduler_t_max=150` and stops at epoch 50, so it is the early slice of a 150-epoch plan rather than a complete 50-epoch cosine run.

The activation and loss branches do not pass the initial screen. LeakyReLU and SiLU reach only 85.84% and 83.36% best validation accuracy, respectively, well below the L0 value of 88.92%. Removing label smoothing and using focal loss produce lower validation-loss values, but their loss objectives differ and should not be compared directly by raw

Table 4 Validation-controlled staged elimination roadmap.

Stage	Budget	Promoted / retained	Eliminated / not used
Structure screen	50 epochs	W4, W6, and CompactV2 all enter Stage B	none; early feasibility check only
Recipe screen	50 epochs	L3 weight decay 10^{-4} , O1 AdamW, O2 Adam	A1 LeakyReLU, A2 SiLU, L1 no label smoothing, L2 focal loss
Stage B	150 epochs	W4 and W6	CompactV2 is dominated by W4
Recipe continuation	150 epochs	L3 weight decay 10^{-4}	O1 AdamW and O2 Adam fall below the L0 baseline
Stage C diagnostic	250 epochs	retained as diagnostic evidence	old continuation curve not used as main evidence
Clean C-lowLR	100 ft epochs	W4, W6, L3-fast, and L3-lowLR enter replay lock	no post-test reselection

**Figure 2** Validation-controlled ablation and selection flow. Architecture screening and recipe ablation are kept separate before both feed into the replay lock.

loss. Under the pre-specified accuracy-first rule, they reach only 86.22% and 85.84%, so they are eliminated.

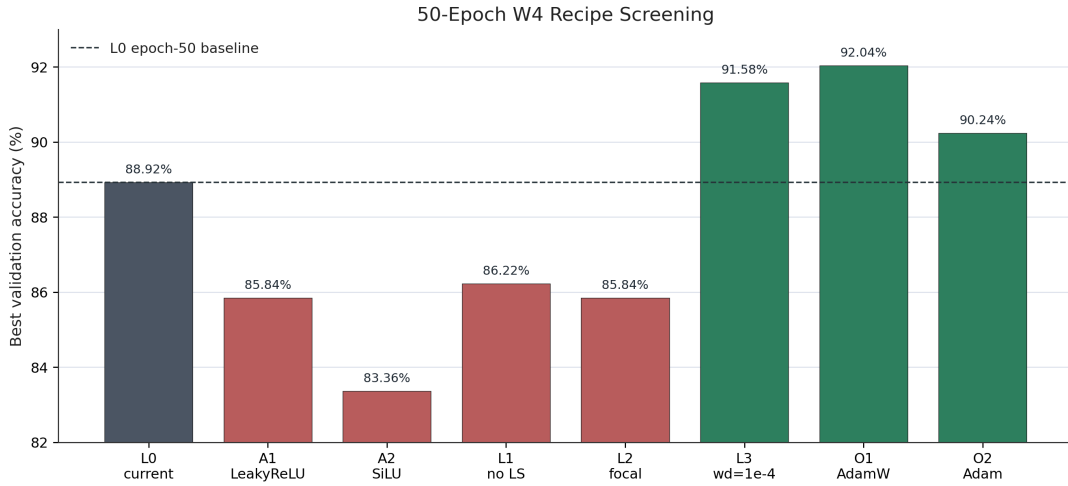
The most promising short-run branches are optimizer and weight decay changes. O1 AdamW reaches 92.04%, O2 Adam reaches 90.24%, and L3 with weight decay 10^{-4} reaches 91.58%. These three branches are therefore promoted to 150-epoch continuation, but this stage supports only a follow-up decision, not a final recipe choice.

3.3 150-Epoch Continuation and Recipe Elimination

The 150-epoch continuation resumes each epoch-50 checkpoint together with its optimizer, scheduler, and random state, so each branch continues the same training trajectory. This design asks a stricter question: if a 50-epoch leader is trained to the same 150-epoch budget as W4 Stage B, does it still outperform L0?

Table 5 W4 recipe 50-epoch screening results.

Run	Factor	Changed setting	Best val acc	Δ vs L0	Decision
L0	Control	current W4 Stage B recipe	88.92%	+0.00 pp	control
A1	Activation	LeakyReLU	85.84%	-3.08 pp	eliminate
A2	Activation	SiLU	83.36%	-5.56 pp	eliminate
L1	Loss	CE without label smoothing	86.22%	-2.70 pp	eliminate
L2	Loss	focal loss, $\gamma = 2$	85.84%	-3.08 pp	eliminate
L3	Regularization	weight decay 10^{-4}	91.58%	+2.66 pp	promote
O1	Optimizer	AdamW, lr 10^{-3}	92.04%	+3.12 pp	promote
O2	Optimizer	Adam, lr 3×10^{-4}	90.24%	+1.32 pp	promote

**Figure 3** W4 recipe 50-epoch screening. Green entries indicate promotion to 150-epoch follow-up, not final selection.

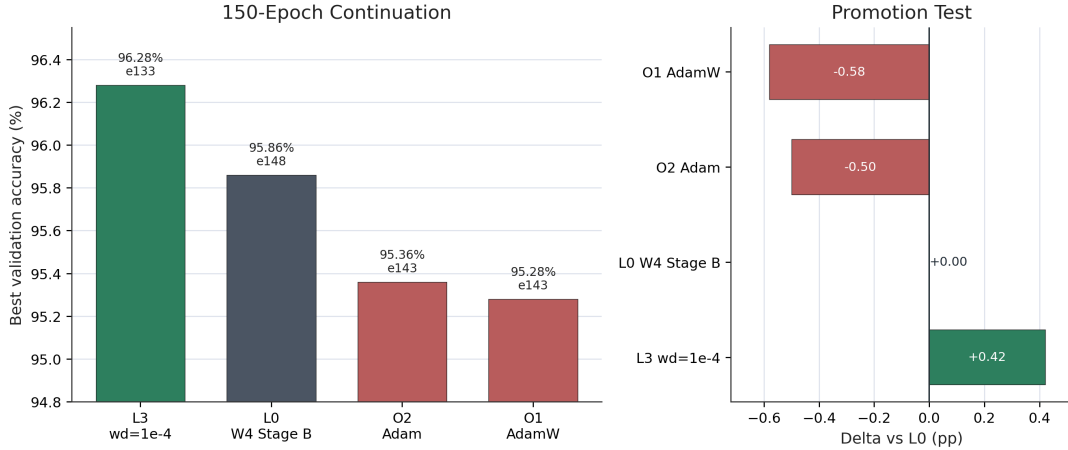
The ranking changes after continuation. AdamW is the strongest branch at 50 epochs, but after 150 epochs it reaches only 95.28%, which is 0.58 pp below the L0 W4 Stage B value of 95.86%. Adam finishes at 95.36%, 0.50 pp below L0. Thus, the short-run optimizer advantage does not translate into a long-run advantage, and O1/O2 are eliminated. The only branch that remains above L0 is L3 with weight decay 10^{-4} , which reaches 96.28% best validation accuracy, 0.42 pp above L0, with best epoch 133.

3.4 Structure Candidates and Stage Progression

Architecture screening runs in parallel with recipe ablation. In the 50-epoch Stage A screen, W6, W4, and CompactResNetV2 are all viable, so all three proceed to Stage B. After the 150-epoch Stage B budget, CompactResNetV2 reaches 94.64% best validation accuracy, below W4’s 95.86%. Its parameter count and memory usage also do not provide enough advantage to offset the accuracy gap, so it is removed from the main candidate

Table 6 W4 recipe 150-epoch continuation results.

Run	Best val acc	Δ vs L0	Best epoch	Final val acc	Decision
L3 weight decay 10^{-4}	96.28%	+0.42 pp	133	96.24%	promote
L0 W4 Stage B	95.86%	+0.00 pp	148	95.84%	baseline
O2 Adam	95.36%	-0.50 pp	143	95.34%	do not promote
O1 AdamW	95.28%	-0.58 pp	143	95.26%	do not promote

**Figure 4** 150-epoch continuation. The early Adam/AdamW advantage disappears; only L3 with weight decay 10^{-4} remains above L0.

set and retained only as development evidence.

W4 and W6 continue to the clean Stage C-lowLR phase. W6 is 0.30 pp above W4 in Stage B and 0.34 pp above W4 in Stage C-lowLR, but it has 13.14M parameters, about 2.25 times the size of W4. For this reason, W6 is kept as the higher-accuracy larger reference, while W4 remains the more efficient Pareto-cost reference.

L3 is a branch from the W4 recipe ablation rather than a new architecture. Its best epoch in the 150-epoch base run is 133, which becomes the L3-fast raw e133 replay entry. In the later low-LR fine-tuning phase, L3 reaches 96.34% best validation accuracy, and the EMA weights at fine-tune epoch 80 match the best validation point; this becomes the L3-lowLR EMA e230 replay entry.

3.5 Stage B, 250-Epoch Diagnostic, and Clean Stage C-lowLR

Stage B shows that W6 has a small but consistent validation-accuracy advantage at a higher cost. W4 reaches 95.86% best validation accuracy, while W6 reaches 96.16%, a 0.30 pp gap. This is not enough to lock W4 as the only candidate, because the larger model may benefit more from longer training or lower learning rates. The experiment therefore proceeds to Stage C.

I also ran a 250-epoch continuation for W4/W6 to check whether simply extending

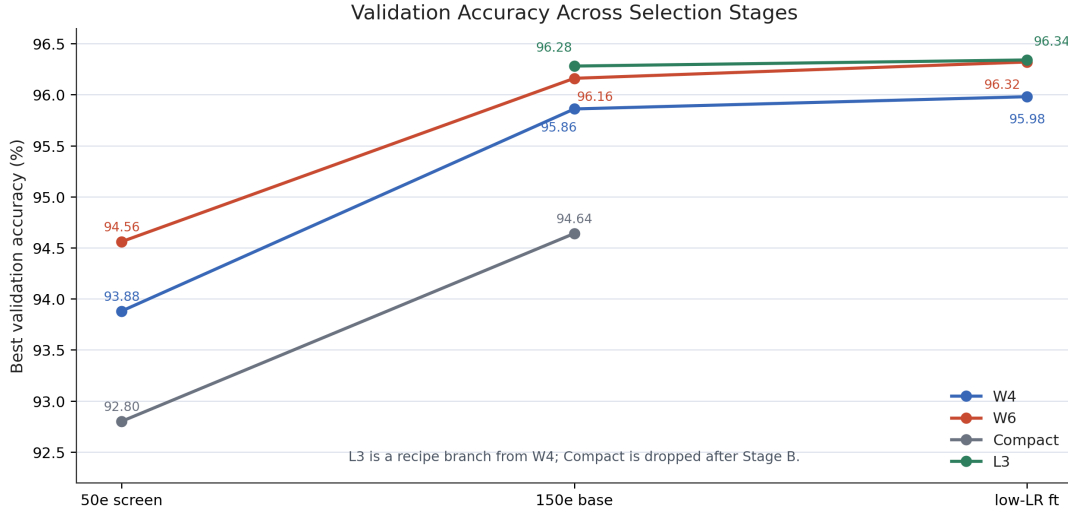


Figure 5 Validation accuracy across selection stages. CompactResNetV2 exits after Stage B, while L3 enters as a W4 recipe branch.

Table 7 Validation-stage Pareto comparison. All results come from `train_dev/val_dev`, not from the official test set.

Run	Best val acc	Epoch	Params	Peak memory	Decision
CompactResNetV2 Stage B	94.64%	138	7.75M	4.98 GiB	dominated by W4; retained only as development evidence
W4 Stage B	95.86%	148	5.85M	3.43 GiB	enters Stage C-lowLR
W6 Stage B	96.16%	142	13.14M	5.56 GiB	enters Stage C-lowLR
L3 150-epoch base	96.28%	133	5.85M	3.43 GiB	replay raw epoch 133
W4 Stage C-lowLR	95.98%	55 ft	5.85M	3.47 GiB	replay raw epoch 205
W6 Stage C-lowLR	96.32%	43 ft	13.14M	5.67 GiB	replay raw epoch 193
L3 Stage C-lowLR	96.34%	80 ft	5.85M	3.47 GiB	replay EMA epoch 230

the training budget would change the Pareto relationship. In that diagnostic run, W4 and W6 reach 96.32% and 96.58% best validation accuracy, respectively. A later audit found that the old continuation curve contained a scheduler-state / high-LR restart artifact, so it is not used as a main-report curve or as a source of replay stop points. Its value is diagnostic: it exposed the scheduler-semantics issue and led to the explicit clean Stage C-lowLR design, which restarts the optimizer and EMA from the 150-epoch base checkpoint under a clearly defined low-learning-rate fine-tuning schedule.

After clean Stage C-lowLR, W4 reaches 95.98% best validation accuracy and W6 reaches 96.32%. W6 remains ahead, but by only 0.34 pp, while parameter count increases from W4’s 5.85M to 13.14M and peak memory rises from 3.47 GiB to 5.67 GiB. The development-stage conclusion is therefore two-sided: W4 is the stronger cost-aware reference, and W6 is the higher-accuracy larger reference.

The later full-train replay does not use the official test set to choose among W4, W6, and L3. Instead, W4, W6, and the two pre-specified L3 checkpoints are all evaluated as locked entries. Therefore, even if one entry is higher on the official test set, the prior selection logic is not changed after the fact.

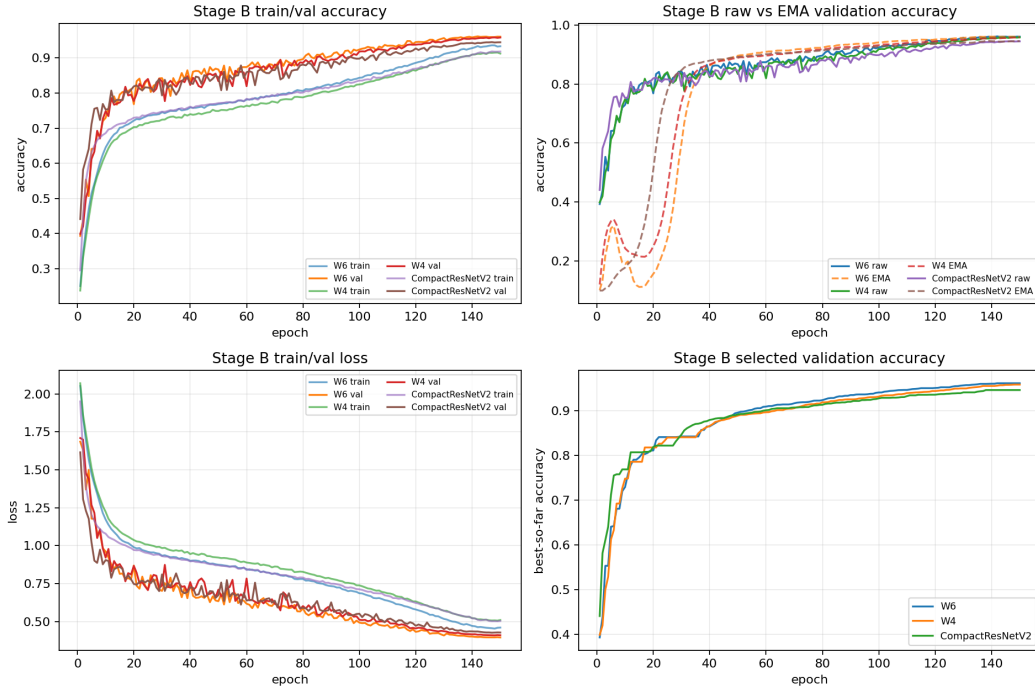


Figure 6 Training and validation curves for Stage B candidates. This stage compares W4, W6, and CompactResNetV2.

3.6 Full-Train Replay Recipe

Full-train replay retrains from scratch on all 50,000 official training images and no longer uses `val_dev` for intermediate selection. The shared recipe has two phases. Phase 1 uses

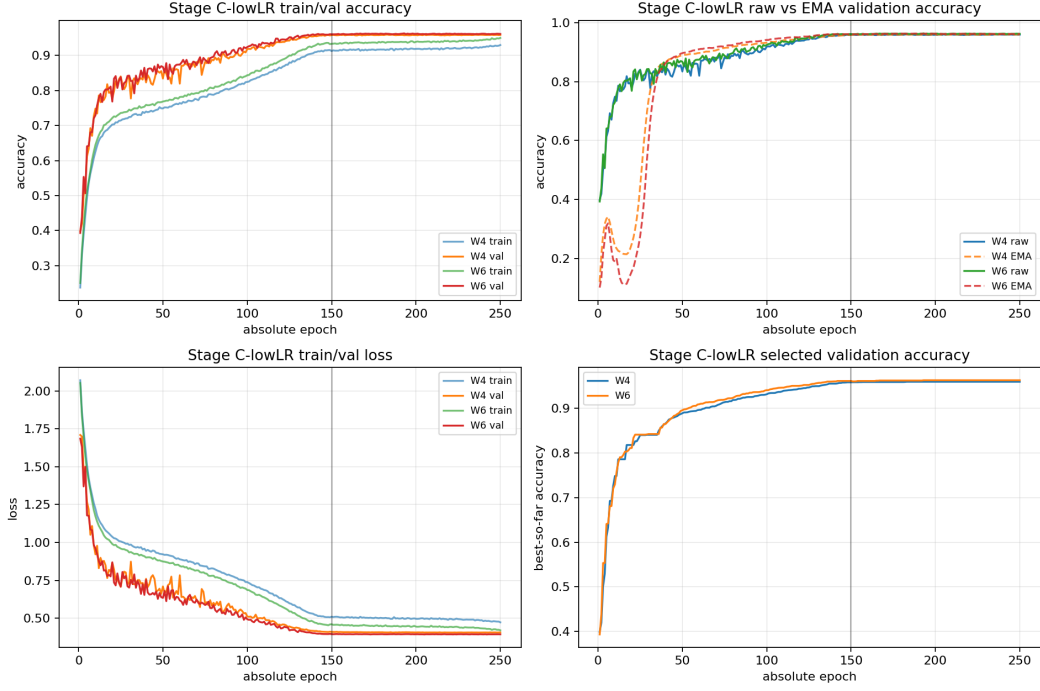


Figure 7 Stage C-lowLR training and validation curves. W4 and W6 continue improving under explicit low-learning-rate fine-tuning; the vertical line marks the transition after epoch 150.

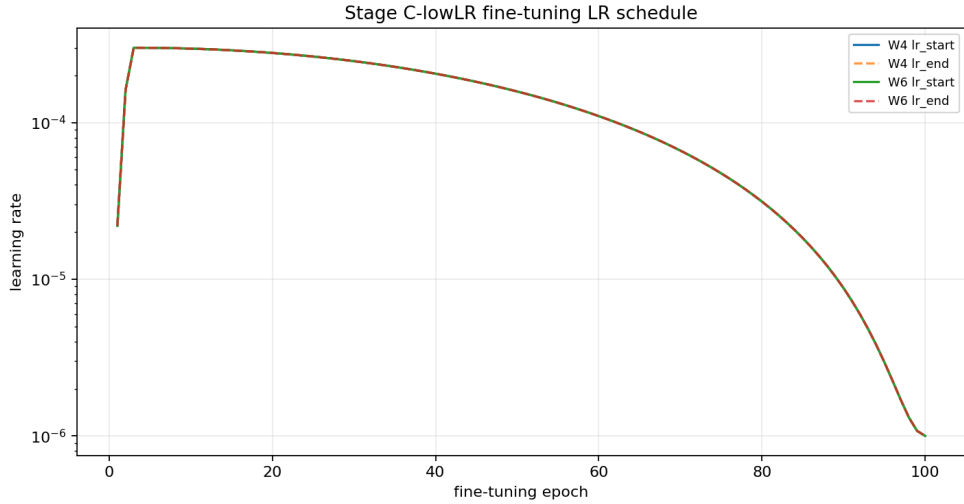


Figure 8 Learning-rate curve for clean Stage C-lowLR. This figure corresponds to the explicit low-learning-rate fine-tuning recipe used for validation-stage Pareto decisions.

150 epochs of base cosine training. Phase 2 restarts the optimizer and EMA, then applies LowLRFineTuneScheduler with `total_epochs=100`, `max_lr=3e-4`, `min_lr=1e-6`, and 3 warmup epochs. The important point is that the low-LR scheduler is not compressed to 43/55/80 epochs; it keeps the same 100-epoch time scale as development, and each model stops at its pre-specified epoch.

Table 8 Full-train replay locked checkpoints. All stop points are fixed before official-test evaluation.

Entry	Global epoch	Weights	Params	Source
W4	205	raw	5.85M	W4 dev-val lowLR fine-tune epoch 55
W6	193	raw	13.14M	W6 dev-val lowLR fine-tune epoch 43
L3-fast	133	raw	5.85M	L3 dev-val best base epoch 133
L3-lowLR	230	EMA	5.85M	L3 dev-val lowLR EMA fine-tune epoch 80

The replay logs contain no validation accuracy and no test accuracy. They record only train-mode loss, train-mode accuracy, learning rate, time, and resource metrics. Train-mode accuracy is affected by RandAugment, Cutout, dropout, and label smoothing, so it is expected to be lower than clean evaluation accuracy. For example, W4 replay ends with 91.57% train-mode accuracy, but its clean train sanity check is 98.73%; L3-lowLR replay has a clean train sanity check of 99.63%.

W4 and L3 have the same parameter count, with average epoch times of 22.55 s and 22.87 s. W6 has 13.14M parameters and an average epoch time of 40.36 s. During replay, I observed low GPU utilization in an early launch setup; after assigning CPU affinity for the three training jobs, the resource logs showed normal GPU utilization.

4 Official Test Results

4.1 Locked Entries

The full-train replay official test contains exactly four entries: W4 raw e205, W6 raw e193, L3-fast raw e133, and L3-lowLR EMA e230. These entries are fixed in `fulltrain\_replay\_test\_lock.json`, and the evaluator checks checkpoint SHA256 hashes. The evaluation uses the clean/eval transform, standard cross-entropy loss, and no horizontal-flip test-time augmentation.

W6 obtains the highest official-test accuracy among the locked replay entries, reaching 96.26%. L3-lowLR EMA reaches 96.10%, slightly above L3-fast raw at 96.02%, but this difference is not used for post-test selection; the two entries answer two different pre-specified questions. W4 replay reaches 95.55%, slightly above the earlier original W4 full-train entry at 95.52%, indicating that the replay protocol does not weaken the W4 baseline.

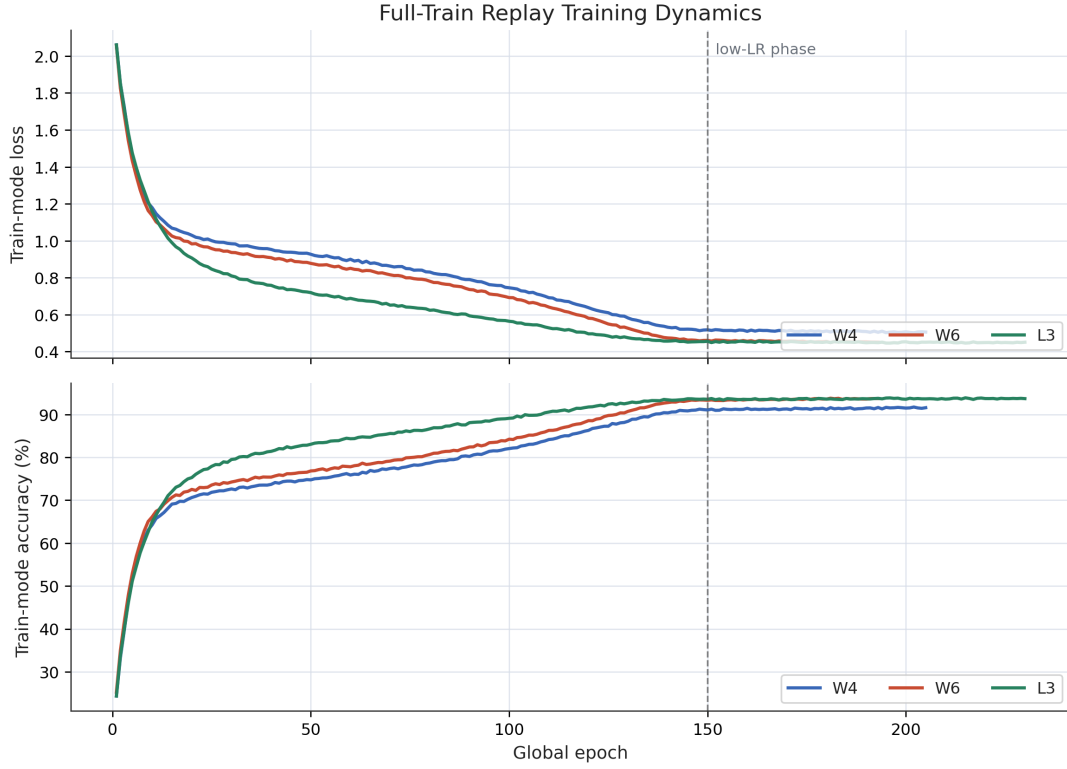


Figure 9 Full-train replay training dynamics. The curves show train-mode metrics; the vertical line marks the transition to explicit low-LR fine-tuning after epoch 150.

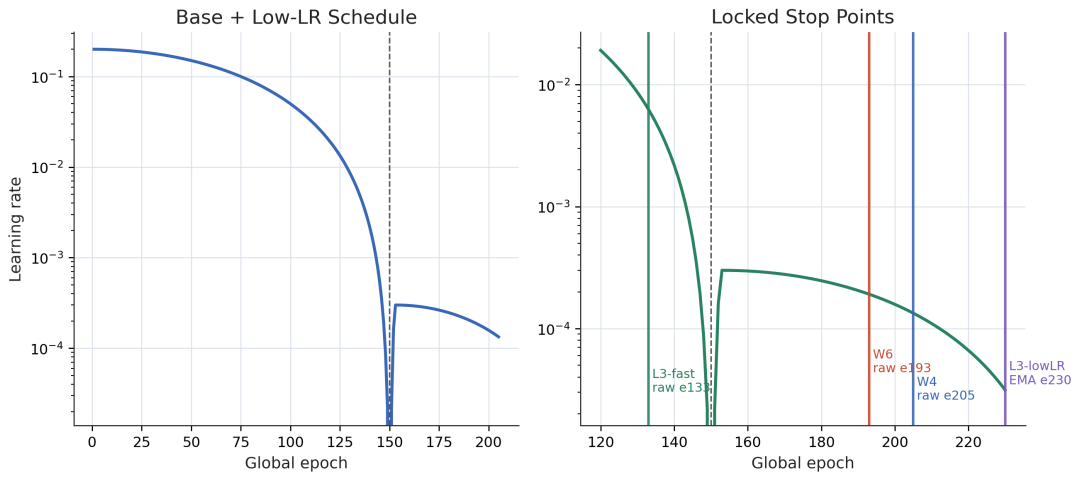


Figure 10 Learning-rate schedule and locked stop points in full-train replay. The low-LR scheduler keeps a 100-epoch time scale, while each model stops at its pre-specified epoch.

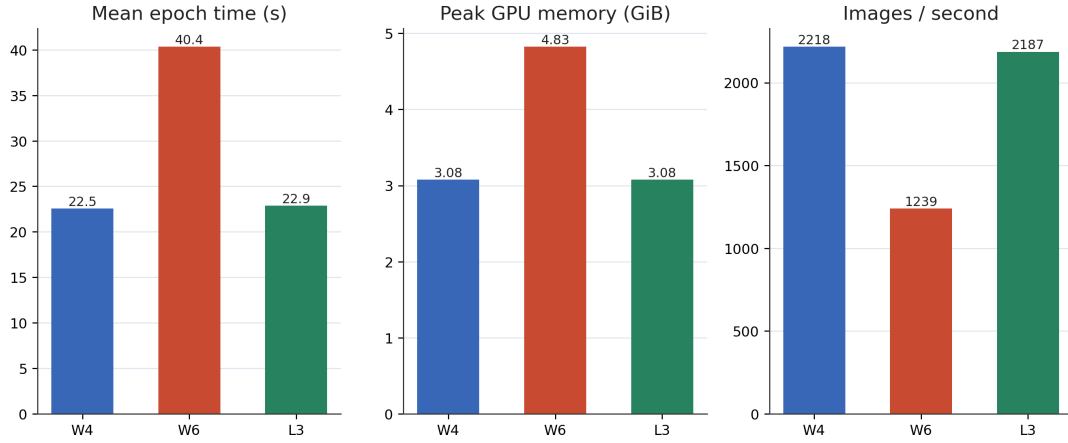


Figure 11 Resource summary for full-train replay. W6 uses more parameters and memory, and its average epoch time is much longer than W4/L3.

Table 9 Full-train replay official-test results. All four entries are locked before testing.

Row	Model	Weights	Params	Test acc	Test error	Test loss
1	W4 full-train replay	raw e205	5.85M	95.55%	4.45%	0.1489
2	W6 full-train replay	raw e193	13.14M	96.26%	3.74%	0.1341
3	L3 full-train fast	raw e133	5.85M	96.02%	3.98%	0.1455
4	L3 full-train lowLR	EMA e230	5.85M	96.10%	3.90%	0.1404

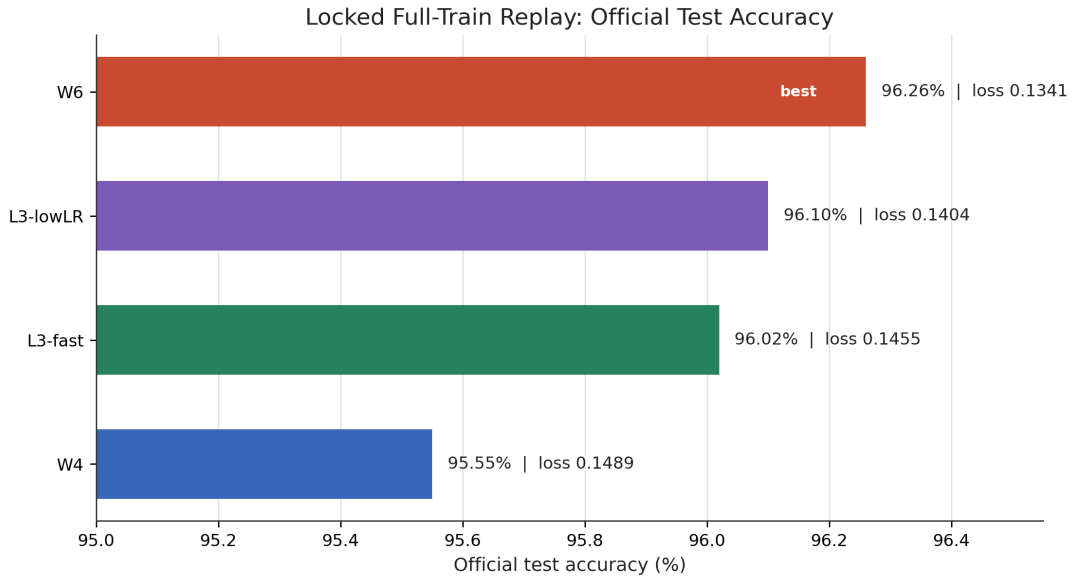


Figure 12 Full-train replay official-test accuracy. W6 is the most accurate model among the four locked replay entries.

4.2 Official-Test Protocol Interpretation

The credibility of the replay result comes mainly from the testing protocol. If TTA, EMA usage, epoch choice, or checkpoint choice were adjusted after seeing the official test result, the test set would become part of model selection. This replay avoids that failure mode: checkpoint paths and hashes are locked before testing; the official-test evaluator accepts only locked entries; and no post-test tuning, post-test training, or checkpoint replacement is performed. The results are frozen in `results/fulltrain_replay_20260611/metrics/fulltrain_replay_official_test_results.csv` and the corresponding JSON files.

This is why L3-fast and L3-lowLR can both appear in the main table, but cannot be compared after the test and reduced to a single chosen entry. Before official-test evaluation, they were defined as different questions: L3-fast tests the early best point of the 150-epoch base recipe, while L3-lowLR tests the low-learning-rate plus EMA extension.

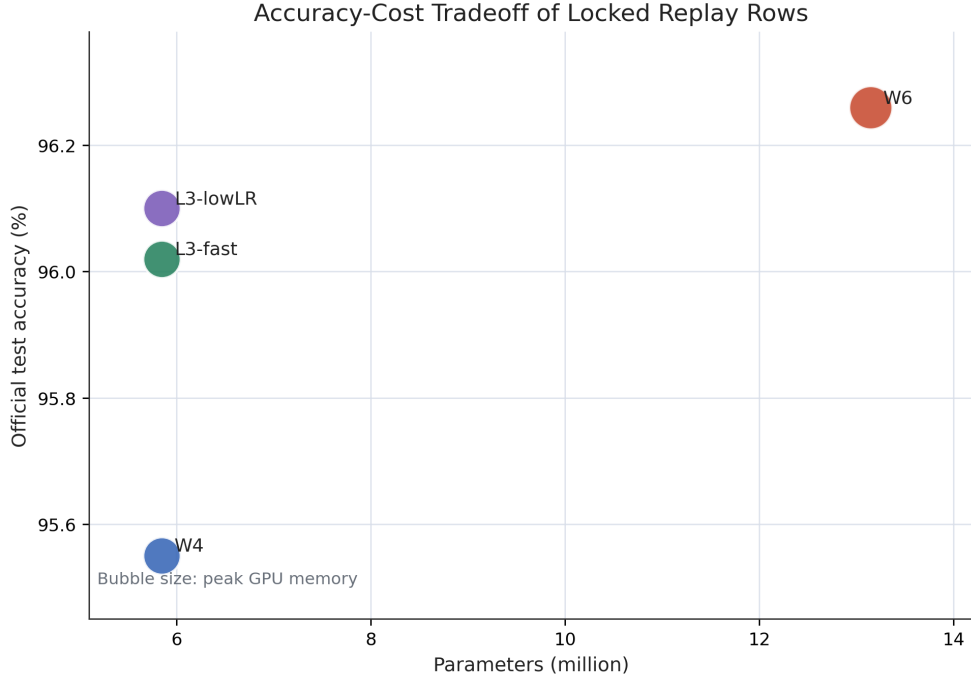


Figure 13 Accuracy-cost tradeoff for locked replay entries. W6 has the highest accuracy, but also the highest parameter and memory cost.

4.3 Classwise Performance and Confusion Matrix

The four replay entries show similar classwise patterns. Automobile, frog, ship, and horse are usually close to or above 97%, while cat and dog remain the main weak classes. W4 reaches 88.1% cat accuracy; W6 improves this to 90.6%; L3-fast and L3-lowLR reach 90.8% and 90.7%, respectively. This pattern is consistent with CIFAR-10’s low resolution and the visual similarity among animal classes.

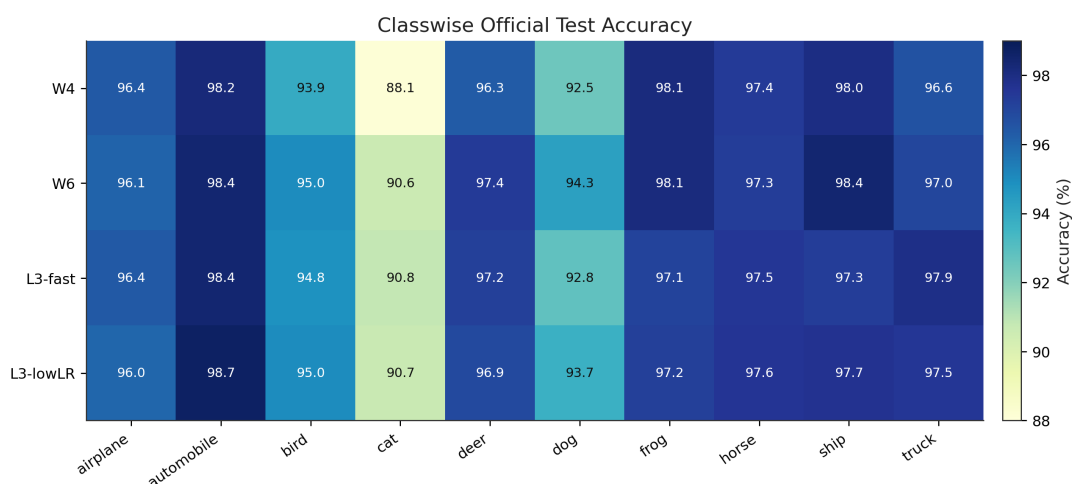


Figure 14 Classwise official-test accuracy for full-train replay. The weakest classes across locked entries are fine-grained animal categories such as cat and dog.

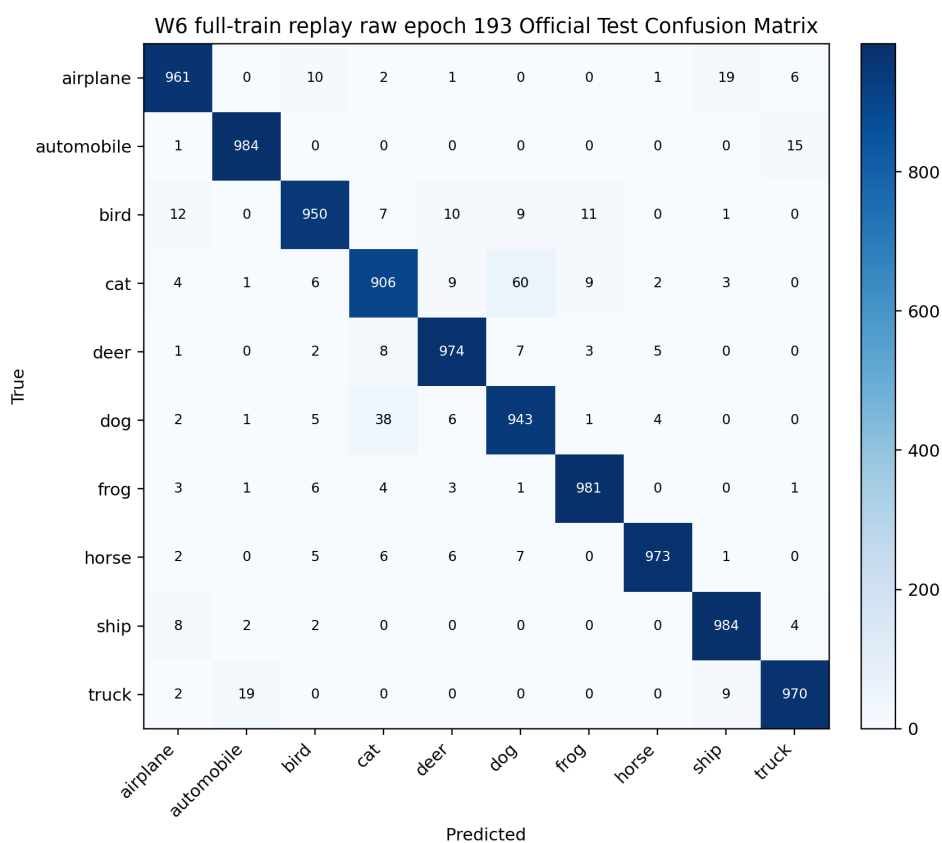


Figure 15 Official-test confusion matrix for W6 replay. W6 is the most accurate locked replay model, and its remaining errors concentrate mainly between visually similar classes.

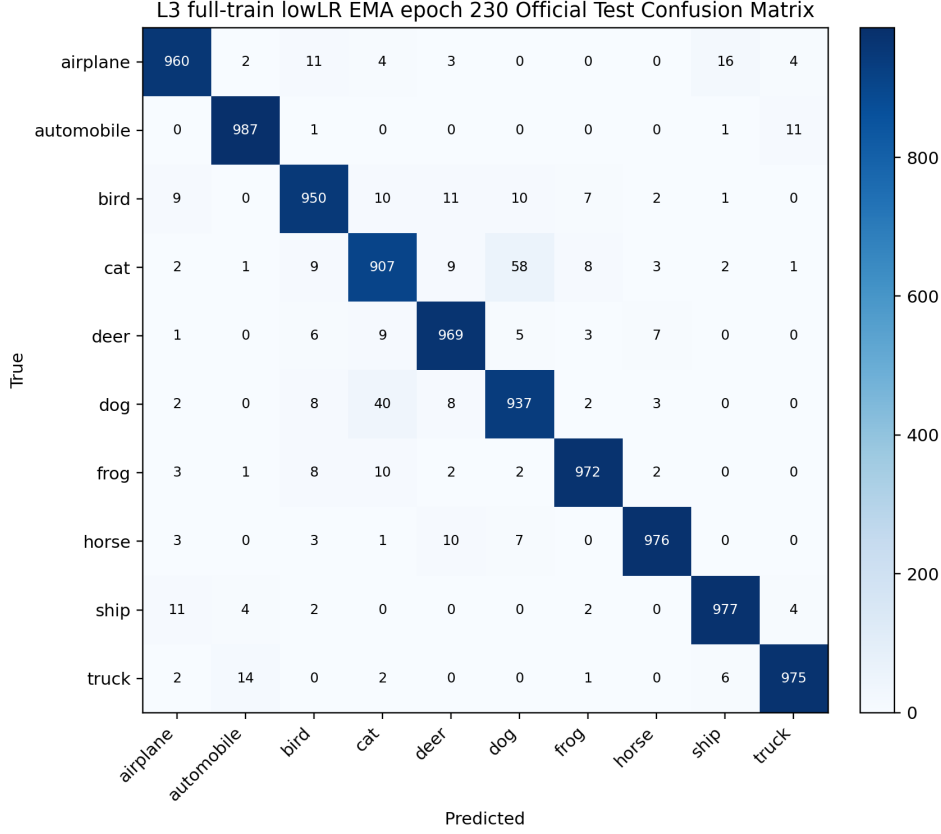


Figure 16 Official-test confusion matrix for L3-lowLR replay. This entry uses EMA weights and reaches 96.10% official-test accuracy.

5 BatchNorm Mini-Research

5.1 Controlled Comparison Design

The BN section analyzes how BatchNorm changes optimization behavior, not only whether it improves final accuracy. The controlled comparison uses VGG-A no BN and VGG-A BN: the backbone layout is the same, and the main difference is the insertion of BatchNorm after convolutional layers. The experiment uses only `train_dev/val_dev`; it does not use the official test set.

BN P0 contains several probes: controlled VGG-A training comparison, validation-threshold reaching, required LR envelope, extended LR stability, scale-invariance probing, directional loss profiling, gradient predictiveness, and gradient Lipschitz estimation. Together, these probes test whether BN changes the optimization surface and local gradient behavior from multiple angles.

5.2 Training Dynamics

The training curves show that the BN model reaches higher validation accuracy under the same epoch budget and crosses the 85%, 88%, and 89% validation-accuracy thresholds

Table 10 Summary of BN P0 results.

Experiment	No-BN	BN	Interpretation
Controlled VGG-A best val acc	88.48%	89.62%	BN improves validation accuracy by 1.14 pp
Epoch reaching 85% val acc	22	19	BN reaches the threshold earlier
Epoch reaching 88% val acc	38	35	BN reaches the threshold earlier
Epoch reaching 89% val acc	not reached	38	BN reaches a threshold not reached by no-BN
Required LR envelope final width	0.9502	0.8563	BN has a narrower final envelope
Expanded LR largest usable LR	0.05	0.20	BN tolerates a higher base LR in short stress runs
Symmetric prediction error	0.6476	0.5171	BN has lower local first-order prediction error
Gradient cosine similarity	0.3553	0.4352	BN has better gradient-direction agreement
Gradient Lipschitz estimate best-val mean	32.1262	32.7714	does not support the claim that BN always lowers the estimate

earlier. The no-BN model never reaches 89% validation accuracy, while the BN model reaches it at epoch 38. Thus, BN’s benefit is visible not only in the final point but also in the training trajectory.

5.3 Learning-Rate Envelope and Stability

The required LR envelope measures the spread of training trajectories across learning rates. The mean width is 0.7671 for BN and 0.7561 for no-BN, so the result does not support the claim that BN uniformly narrows the envelope over the full run. However, BN has lower final width and max width, 0.8563 and 1.1370, compared with no-BN values of 0.9502 and 1.2806. The more defensible interpretation is that BN improves trajectory stability in some phases of this setting, but not under every learning-rate condition.

5.4 Scale Invariance

One important BN property is that train-mode batch statistics can offset activation-scale changes caused by rescaling earlier weights. The scale-invariance probe rescales convolutional weights and measures whether loss and predictions change. For no-BN, both loss and predictions change substantially. For train-mode BN, the loss change is about 1.1×10^{-5} and the argmax flip rate is 0. The figure also includes BN eval-mode running-statistics results as a caveat: eval-mode BN does not preserve the same scale

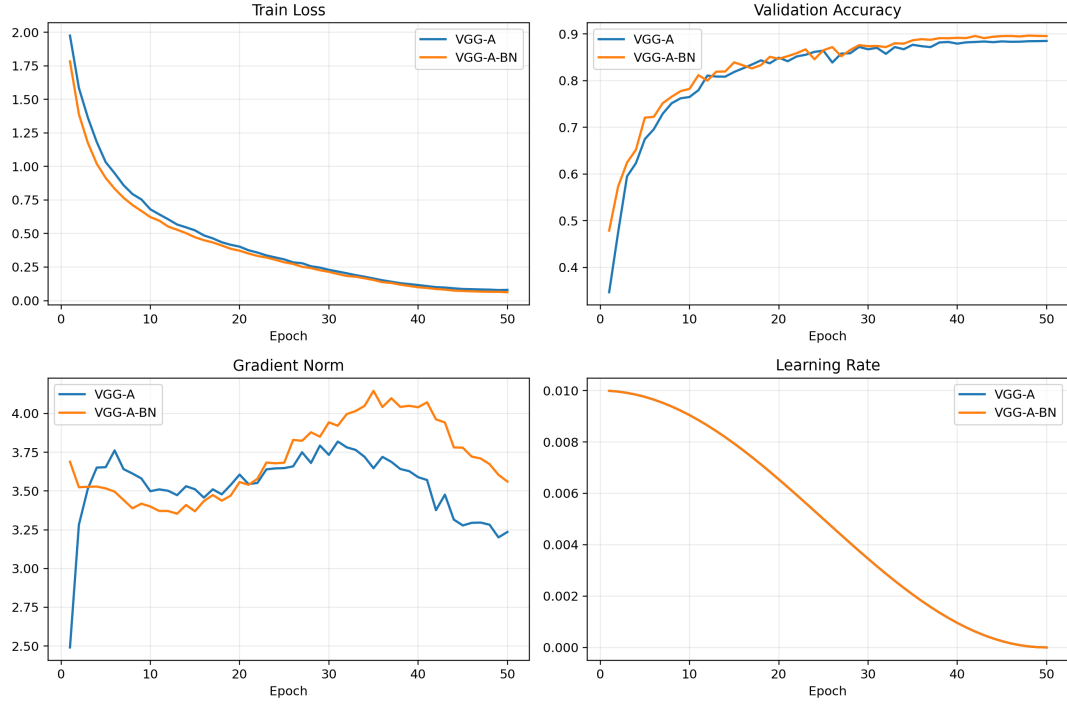


Figure 17 BN P0 training-dynamics dashboard. The BN model improves validation accuracy and reaches key thresholds earlier than the no-BN model.

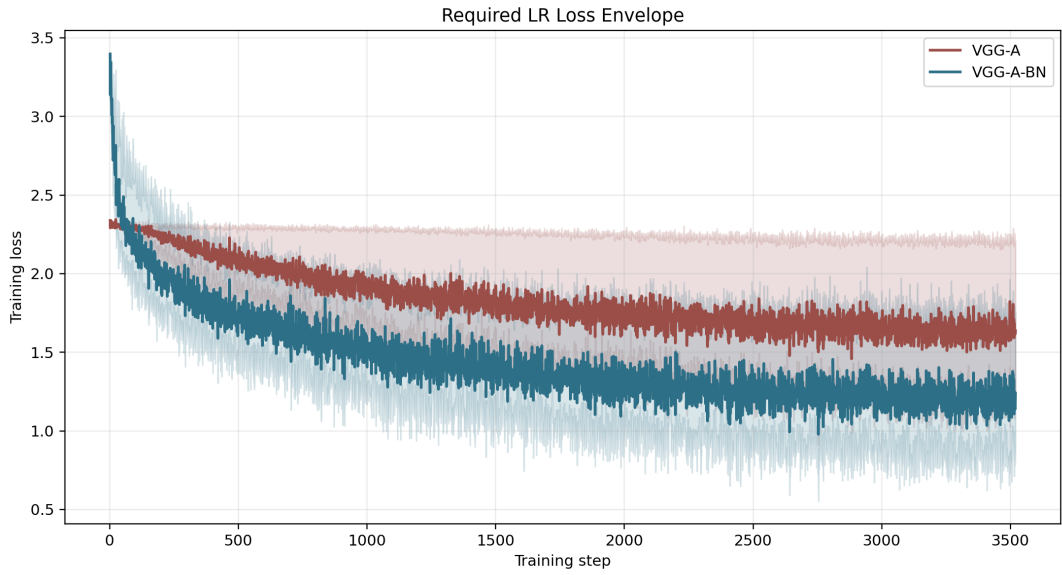


Figure 18 BN required loss envelope. BN has smaller final and maximum widths, but not a smaller mean width.

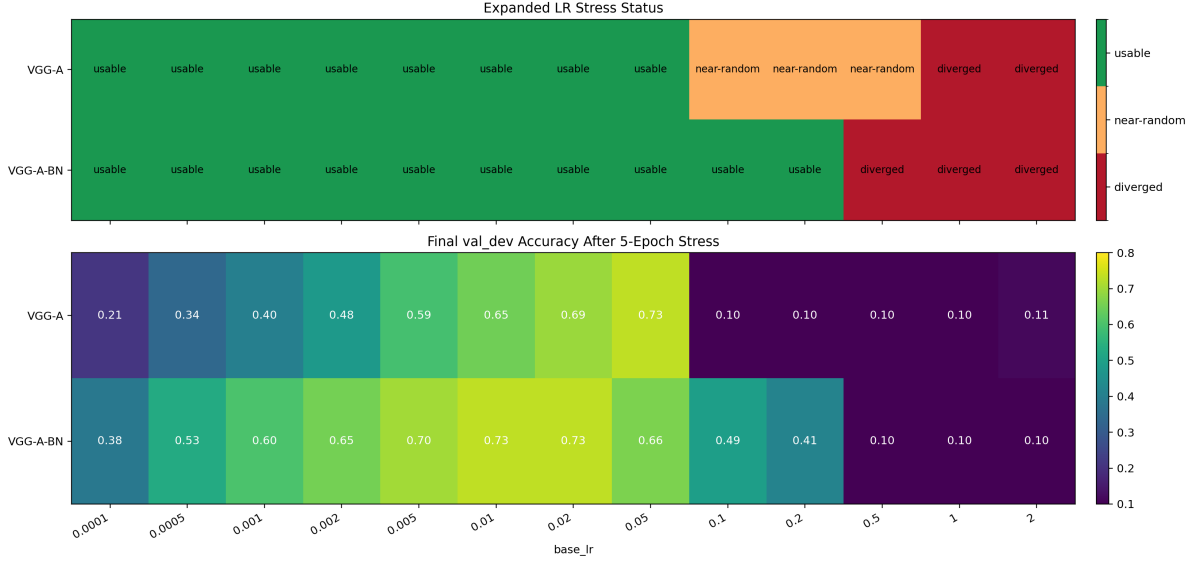


Figure 19 BN learning-rate stability heatmap. The top panel uses three states: usable, near-random, and diverged; the bottom panel reports continuous final validation accuracy. In the expanded short-run stress test, the largest usable base LR is 0.05 for VGG-A no BN and 0.20 for VGG-A BN.

invariance. The supported conclusion is therefore restricted to train-mode batch-statistics reparameterization.

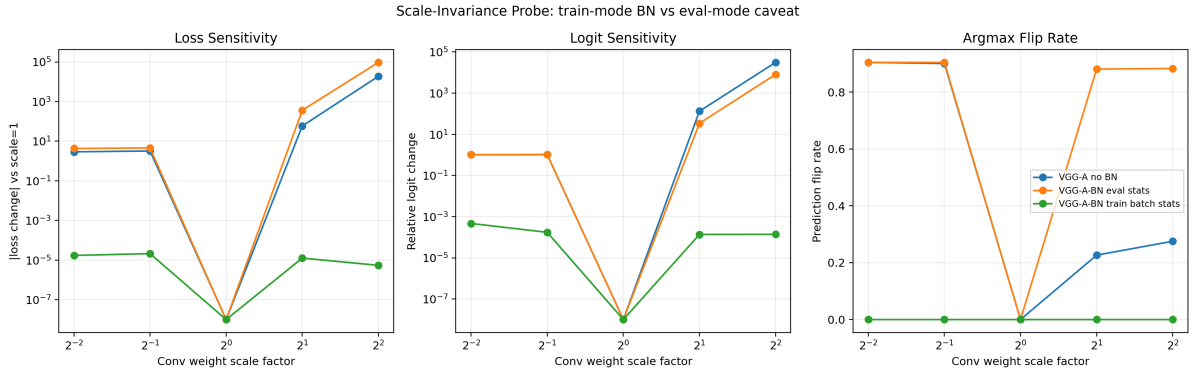


Figure 20 BN scale-invariance probe. Train-mode BN keeps loss, logits, and predictions nearly unchanged under convolution-weight rescaling; no-BN and eval-mode running statistics remain sensitive to rescaling. The argmax flip rate is a discrete prediction metric and can saturate under extreme scales.

5.5 Directional Profile and Gradient Predictiveness

The directional loss profile applies local perturbations along the gradient direction and a random direction. Along d_{grad} , BN has a mean absolute loss delta of 0.3503, below no-BN's 0.4370. Along d_{random} , BN has 0.000072, below no-BN's 0.000234. In these measured local directions, BN therefore produces smaller loss changes.

The gradient-predictiveness probe measures how well a first-order gradient estimate predicts local loss changes. Raw relative error can be inflated when the actual loss change is very small, so the report uses symmetric prediction error as the more robust metric. BN has lower symmetric prediction error, 0.5171 versus 0.6476 for no-BN. BN also has higher gradient cosine similarity, 0.4352 versus 0.3553, and lower relative gradient change, 1.0037 versus 3.0477. Together, these metrics support the claim that BN improves local linear predictiveness and gradient-direction consistency in this probe.

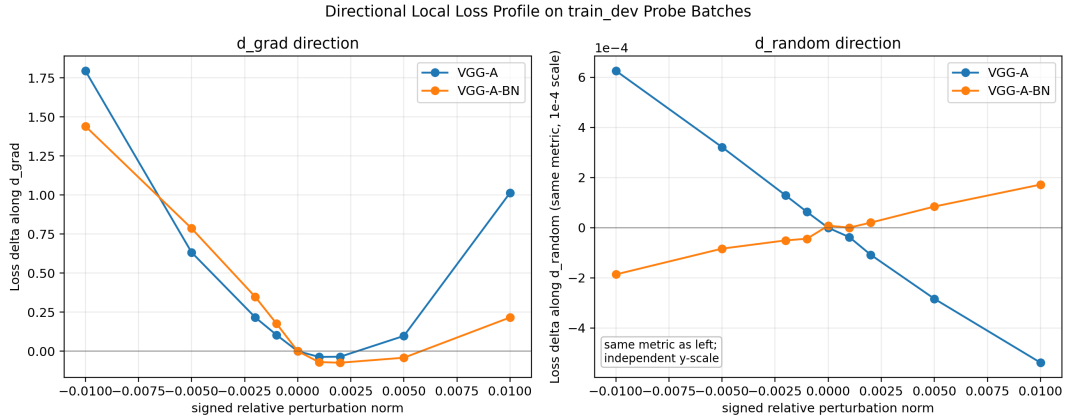


Figure 21 BN directional loss profile. Both panels use loss delta as the metric, but the directions differ: the left panel follows the gradient direction, while the right panel follows a random direction. The right y-axis uses an independent 10^{-4} -scale range, so its absolute height should not be compared directly with the left panel.

5.6 Supported Claims and Non-Claims

The BN conclusions are deliberately limited to what the experiments support. The supported claims are: BN improves controlled VGG-A validation accuracy; BN reaches several validation-accuracy thresholds earlier; BN tolerates a higher usable base LR in the expanded short-run LR stress test; BN is nearly invariant to convolution-weight rescaling under train-mode batch statistics; and BN improves several local gradient-predictiveness metrics.

The experiments do not support stronger claims that BN narrows the loss envelope at all learning rates, that a short LR stress test gives a universal maximum stable learning-rate rule, that BN necessarily lowers the gradient Lipschitz estimate, or that one VGG-A/CIFAR-10 setting generalizes to all models and datasets. Stating these boundaries prevents local evidence from being overstated as a general law.

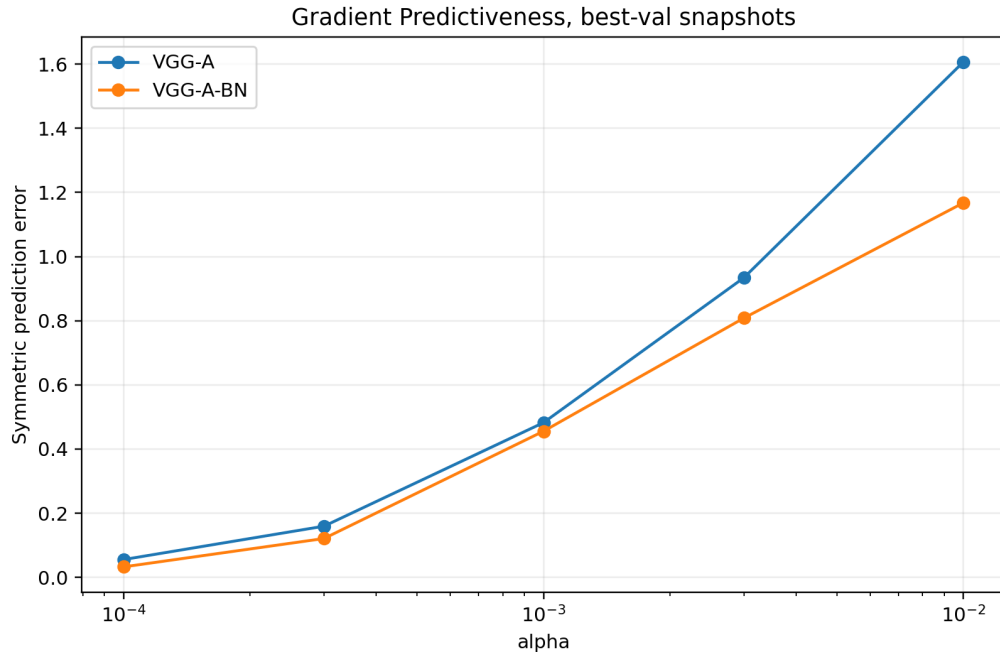


Figure 22 BN gradient predictiveness versus alpha. With symmetric prediction error, BN has lower local first-order prediction error.

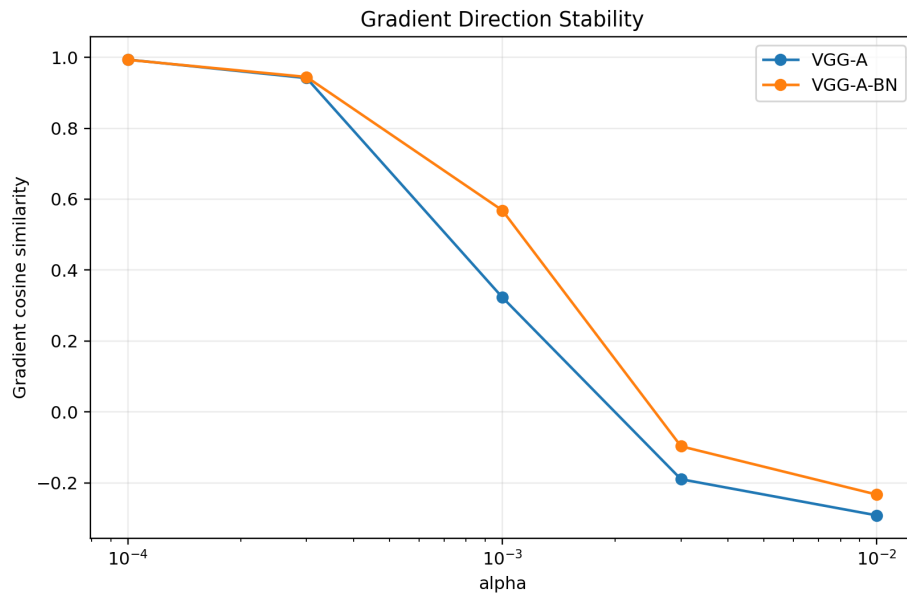


Figure 23 BN gradient cosine similarity. BN shows better agreement in gradient direction.

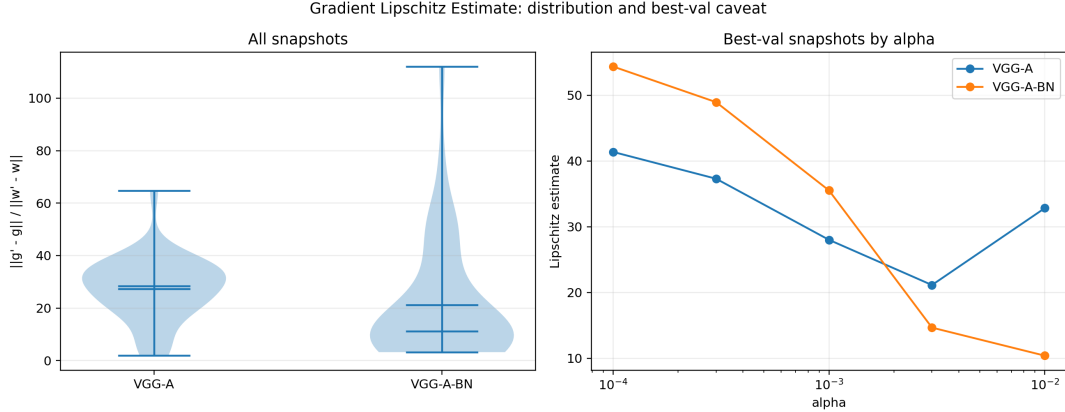


Figure 24 BN gradient Lipschitz estimate. The all-snapshot distribution and the best-validation snapshot do not give the same ordering, so this figure is used as cautious non-claim evidence.

6 Error Analysis and Discussion

6.1 Where the Replay Models Still Fail

The locked replay entries have official-test error rates between 3.74% and 4.45%. W6 has the lowest error at 3.74%, followed by L3-lowLR at 3.90%, L3-fast at 3.98%, and W4 at 4.45%. Classwise accuracy shows that cat and dog remain clearly weaker than automobile, ship, horse, and several other classes. This pattern is plausible for three reasons: CIFAR-10 images are only 32×32 , animal classes have similar local textures and outlines, and object-background ratios vary enough that some examples may encourage background-based cues.

The confusion matrices confirm that many errors occur between visually similar classes. These failures are not merely signs of under-training; they also reflect ambiguity in the dataset. Further improvement could come from stronger augmentation, longer training, larger models, ensembling, or test-time augmentation, but none of these changes is added after official-test evaluation because the replay protocol is already locked.

6.2 How to Interpret W4, W6, and L3

If only official-test accuracy is considered, W6 is the best of the four replay entries at 96.26%. However, W6 has 13.14M parameters, while W4 and L3 have 5.85M; W6 also uses 4.83 GiB peak memory in replay and averages 40.36 s per epoch, clearly above W4/L3. W6 should therefore be interpreted as the highest-accuracy larger model, not as an unconditional winner under every cost criterion.

The two L3 entries answer different questions. L3-fast raw e133 reaches 96.02% without using the low-LR replay endpoint, while L3-lowLR EMA e230 reaches 96.10%. The 0.08 pp gap shows that the early L3 recipe point is already strong and that low-LR

+ EMA adds a small extra gain. Since both entries are pre-specified before testing, the higher test value cannot be used to discard the other entry after the fact.

6.3 Protocol Lessons

The main experimental lesson is that a strict protocol matters more than any single accuracy number. The development stage can use the validation set for search, but once the replay lock is created, the official test set should not drive any further adjustment. Full-train replay is useful because it maps validation-selected decisions onto the full 50,000-image training set while preserving a clear “locked before test” boundary.

A second lesson is that results should be interpreted together with cost. W6 has the highest official-test accuracy, but it is also the slowest and largest model. VGG-A has more parameters than W4/L3 but performs much worse than the PreActResNet replay entries. BN adds almost no parameters, yet changes optimization behavior substantially. These observations show why model comparison should not be reduced to one accuracy column.

A third lesson is that mechanism analysis should include non-claims. BN clearly improves VGG-A optimization in this project, and the expanded short-run LR stress test shows that BN tolerates a higher usable base LR. However, the experiments do not support several common overgeneralizations: the mean required-LR envelope width does not decrease, and the best-validation Lipschitz-estimate mean does not decrease. Reporting only positive claims would hide the actual boundaries of the evidence.

7 Conclusion

This project presents a validation-controlled CIFAR-10 classification study. During development, a 45,000/5,000 train/validation split is used for model selection and stop-point decisions. The selected recipes are then retrained on all 50,000 official training images in full-train replay. Official-test evaluation is performed only after all four checkpoints are locked, using a clean/eval transform and no horizontal-flip TTA. Among the locked replay entries, W6 obtains the highest official-test accuracy at 96.26% with 3.74% error; L3-lowLR reaches 96.10%; L3-fast reaches 96.02%; and W4 reaches 95.55%.

The model comparison shows a clear progression. SimpleCNN provides an 86.72% low-complexity baseline. VGG-A no BN and VGG-A BN reach 87.85% and 89.17%, respectively, showing that BN helps the VGG-A control setting. The PreActResNet replay entries achieve much higher accuracy with parameter counts below or comparable to VGG-A. W6 is the highest-accuracy larger model, while W4/L3 reach 95.55%–96.10% with only 5.85M parameters, giving better parameter efficiency.

The BN mini-study supports a cautious but clear conclusion: in the controlled VGG-A/CIFAR-10 setting, BN improves validation accuracy, convergence speed, ex-

panded short-run LR robustness, train-mode scale invariance, and several local gradient-predictiveness metrics. It does not support stronger claims that BN is always smoother or that short stress tests generalize to arbitrary training budgets. Overall, the main contribution of the project is not a single model but a complete experimental workflow from development validation to model selection, replay locking, and frozen official-test evaluation.

References

- [1] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.
- [2] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [3] S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In Proceedings of the International Conference on Machine Learning, 2015.
- [4] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry. How Does Batch Normalization Help Optimization? In Advances in Neural Information Processing Systems, 2018.